Лабораторная работа №3

Задачи:

1. Развернуть кластер Kubernetes
2. Написать код, который собирает значения нагрузки на CPU и RAM в течении 5 минут и выводит график относительно собранных данных

Ход работы:

1. Для виртуализации рекомендуется использовать VMWare Workstation Pro.
2. Для развертки кластера создайте две виртуальные машины (далее -ВМ). Первая ВМ будет «master» и иметь образ с графическим интерфейсом ubuntu v.22.04 для удобства работы (https://ubuntu.com/download/desktop). Вторая ВМ «worker1» будет иметь образ сервера (https://ubuntu.com/download/server). Необходимо пройти полный процесс установки для обеих машин. Имена пользователей делайте одинаковыми.
3. Вся работа будет происходить с «master». Для быстрого перехода и работы на «worker1» необходимо на «master» настроить подключение по SSH. *Для копирования с вашего устройства на «master» используйте сочетания клавиш Ctrl + C, Ctrl + V для устройства и Ctrl + Shift + C, Ctrl + Shift + V для «master». Команды для вставки выделены серым цветом.*

```
user@master:~$ cd /home
```

```
user@master:~$ ssh-keygen
```

```
user@master:~$ ssh-copy-id user@192.168.196.132
```

`user@master:~$ ssh 'user@192.168.196.132'` (команда для подключения к worker1, чтобы выйти необходимо нажать Ctrl + D)

Для подключения к «worker1», ВМ «worker1» должна быть запущена.

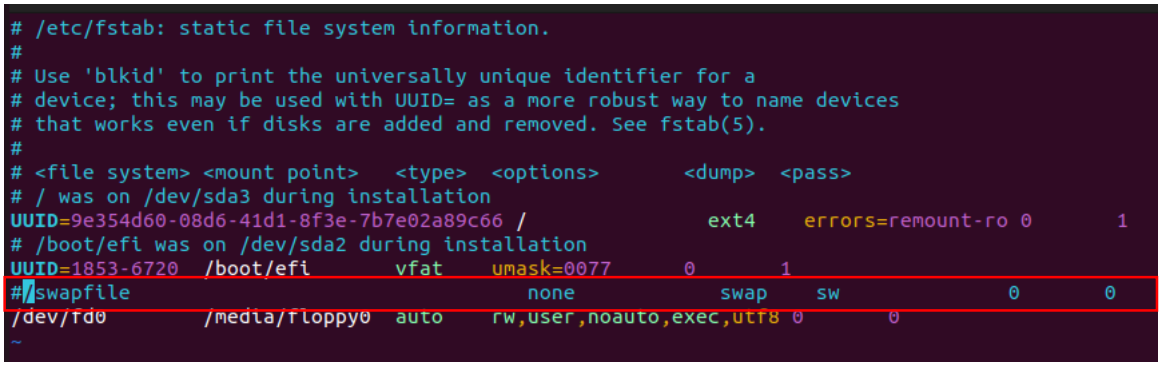
192.168.196.132– ip worker1 (можно посмотреть с помощью команды `sudo ip addr show`)

Далее действия будут выполнятся как для «master» так и для «worker1» чтобы переключатся между машинами используйте `ssh 'user@192.168.196.132'`

4. Отключите подкачку swap на “master”

```
user@master:~$ sudo vi /etc/fstab
```

Комментируем строку показанную на скрине (изначально она без #).



```
user@master:~$ sudo swapoff -a
```

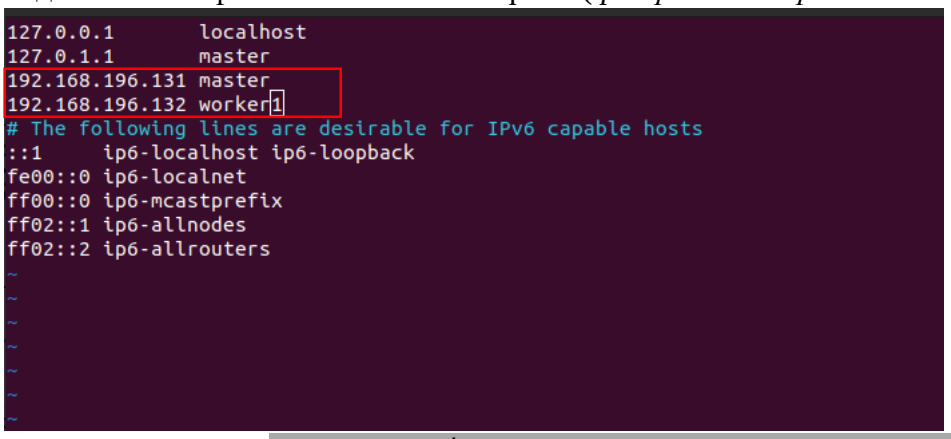
Затем тоже самое делаем на «worker1». Если в файле `vi /etc/fstab` нет строки как на скрине то просто выполняем команду `sudo swapoff -a`

5.
```
user@master:~$ sudo hostnamectl set-hostname "master"
user@master:~$ exec bash
```

На «worker1» команда

```
user@master:~$ sudo hostnamectl set-hostname "worker1"
user@master:~$ exec bash
```

6. На «master» вводим команду
```
user@master:~$ sudo vim /etc/hosts
```
И добавляем в файл записи как на скрине (*ip адреса смотрите на своих машинах*)



7.
```
user@master:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
>overlay
>br_netfilter
>EOF
```
8.
```
user@master:~$ sudo modprobe overlay
user@master:~$ sudo modprobe br_netfilter
```
9.
```
user@master:~$ cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
> net.bridge.bridge-nf-call-iptables  = 1
> net.bridge.bridge-nf-call-ip6tables = 1
> net.ipv4.ip_forward                 = 1
>EOF
```
10.
```
user@master:~$ sudo sysctl -system
```
11. Пункты 7-10 повторяем на «worker1»
12. Устанавливаем `kubelet kubeadm kubectl`
```
user@master:~$ sudo -i
root@master:/home/user#  curl -s
https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-key add
-
root@master:/home/user# cat <<EOF
>/etc/apt/sources.list.d/kubernetes.list
> deb http://apt.kubernetes.io/ kubernetes-xenial main
>EOF
```

```
root@master:/home/user# echo ‘deb http://apt.kubernetes.io/ kubernetes-xenial main’ | sudo tee /etc/apt/sources.list.d/kubernetes.list
root@master:/home/user# apt-get update
root@master:/home/user# apt-get install -y kubelet kubeadm kubectl
```

13. Устанавливаем Докер

```
root@master:/home/user# sudo apt install docker.io
root@master:/home/user# sudo mkdir /etc/containerd
root@master:/home/user# sudo sh -c "containerd config default > /etc/containerd/config.toml"
root@master:/home/user# sudo sed -i 's/ SystemdCgroup = false/ SystemdCgroup = true/' /etc/containerd/config.toml
root@master:/home/user# sudo systemctl restart containerd.service
root@master:/home/user# sudo systemctl restart kubelet.service
root@master:/home/user# sudo systemctl enable kubelet.service
```

14. На «worker1» повторяем пункты 12 и 13
15. На «master» выполняем следующие команды

```
user@master:~$ sudo kubeadm config images pull
user@master:~$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.196.131
```

192.168.196.131 – ip адрес «master» (у Вас свой)

```
user@master:~$ mkdir -p $HOME/.kube
user@master:~$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
user@master:~$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
user@master:~$ kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/tigera-operator.yaml
user@master:~$ curl https://raw.githubusercontent.com/projectcalico/calico/v3.26.1/manifests/custom-resources.yaml -O
user@master:~$ sed -i 's/cidr: 192\.168\.0\.0\/16/cidr: 10.244.0.0\/16/g' custom-resources.yaml
user@master:~$ kubectl create -f custom-resources.yaml
```

16. На «worker1» выполняем команду которая выходит в конце выполнения команды `sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.196.131`

Пример: user@worker1:~$ sudo kubeadm join 192.168.196.131:6443 --token 7f8dg2.242f14l0q536ild9 --discovery-token-ca-cert-hash sha256:f2edb9abf6bb3f8619a5048e987343909c9651cb24d4e7e63e40287b0d312c3e

17. Относительно второго задания Вам необходимо написать код на любом языке программирования (рек. python). Код должен собирать значения нагрузки на CPU и RAM в течении 5 минут и выводит график относительно собранных данных. Пример результата работы программного кода

	Standard	Standard	Standard
5	worker1	24.459	1935.684
6	worker1	24.459	1935.684
7	worker1	24.694	1936.898
8	worker1	24.694	1936.898
9	worker1	10.002	1958.32
10	worker1	10.002	1958.32
11	worker1	31.672	1933.637
12	worker1	31.672	1933.637

